

DecisionIQ Master Handbook

CHAPTER 4: Advanced SQL & Predictive Machine Learning

4.1 Advanced Analytics in SQL

To keep DecisionIQ fast and scalable, we run heavy calculations directly in the database using SQL. This reduces network overhead by aggregating data at the storage layer rather than downloading millions of raw rows to our web server.

We use three core SQL techniques to calculate business performance:

4.1.1 Customer Cohort Retention Analysis

A **Cohort** is a group of customers who share a common characteristic (e.g., they made their first purchase in the same month). **Retention Analysis** tracks what percentage of a cohort continues to buy in subsequent months.

```
WITH CustomerCohorts AS (  
    -- Identify the acquisition month for each customer  
    SELECT  
        customer_id,  
        STRFTIME('%Y-%m', acquisition_date) AS cohort_month  
    FROM customers  
)  
CustomerActivity AS (  
    -- Find all distinct months when a customer placed an order  
    SELECT DISTINCT  
        customer_id,  
        STRFTIME('%Y-%m', order_date) AS activity_month  
    FROM orders  
    WHERE status NOT IN ('Cancelled')  
)  
CohortSizes AS (  
    -- Count total unique customers in each cohort  
    SELECT  
        cohort_month,  
        COUNT(customer_id) AS cohort_size  
    FROM CustomerCohorts  
    GROUP BY 1  
)  
CohortRetention AS (  
    -- Calculate month index difference (Retention Month 0, 1, 2... 12)  
    SELECT  
        cc.cohort_month,  
        ((CAST(SUBSTR(ca.activity_month, 1, 4) AS INTEGER) -  
        CAST(SUBSTR(cc.cohort_month, 1, 4) AS INTEGER)) * 12) +  
        (CAST(SUBSTR(ca.activity_month, 6, 2) AS INTEGER) -
```

```

CAST(SUBSTR(cc.cohort_month, 6, 2) AS INTEGER)) AS month_number,
    COUNT(DISTINCT cc.customer_id) AS active_customers
FROM CustomerCohorts cc
JOIN CustomerActivity ca ON cc.customer_id = ca.customer_id
GROUP BY 1, 2
)
SELECT
    cr.cohort_month,
    cs.cohort_size,
    cr.month_number,
    cr.active_customers,
    ROUND((cr.active_customers * 100.0) / cs.cohort_size, 2) AS retention_pct
FROM CohortRetention cr
JOIN CohortSizes cs ON cr.cohort_month = cs.cohort_month
ORDER BY cr.cohort_month DESC, cr.month_number ASC;

```

4.1.2 RFM Customer Segmentation

We segment customers based on their purchasing habits using three metrics:

1. **Recency (R):** Days since their last order.
2. **Frequency (F):** Total number of orders placed.
3. **Monetary (M):** Total spent on successful orders.

We rank customers into quintiles (1 to 5) for each metric using the `NTILE(5)` window function:

```

WITH CustomerMetrics AS (
    SELECT
        customer_id,
        (SELECT JULIANDAY(MAX(order_date)) FROM orders) -
        JULIANDAY(MAX(order_date)) AS recency_days,
        COUNT(order_id) AS frequency,
        SUM(total_amount) AS monetary_value
    FROM orders
    WHERE status <> 'Cancelled'
    GROUP BY customer_id
)
SELECT
    customer_id,
    NTILE(5) OVER (ORDER BY recency_days DESC) AS r_score,
    NTILE(5) OVER (ORDER BY frequency ASC) AS f_score,
    NTILE(5) OVER (ORDER BY monetary_value ASC) AS m_score
FROM CustomerMetrics;

```

By adding these scores, we identify customer segments (e.g., a customer with 5-5-5 is a "Champion," while 1-5-5 is "At-Risk").

4.1.3 Recursive CTE for Sales Rollups

In large organizations, employees report to managers in a hierarchical tree. To calculate the total sales generated under each manager's reporting line, we use a **Recursive Common Table Expression (CTE)**:

```
WITH RECURSIVE OrgHierarchy AS (  
  -- Anchor Member: Find the CEO (who reports to no one)  
  SELECT  
    employee_id,  
    first_name || ' ' || last_name AS employee_name,  
    role,  
    reports_to,  
    1 AS level_depth  
  FROM employees  
  WHERE reports_to IS NULL  
  
  UNION ALL  
  
  -- Recursive Member: Join employees to their managers  
  SELECT  
    e.employee_id,  
    e.first_name || ' ' || e.last_name AS employee_name,  
    e.role,  
    e.reports_to,  
    h.level_depth + 1 AS level_depth  
  FROM employees e  
  JOIN OrgHierarchy h ON e.reports_to = h.employee_id  
)  
SELECT * FROM OrgHierarchy;
```

4.2 Machine Learning Pipelines

While SQL is great for analyzing historical data, we use **Machine Learning** to forecast future performance and predict risks.

DecisionIQ ML Scoring Flow
1. Feature Prep: Prepare SQL metrics (CSAT, RFM, Tickets)
2. Modeling: Train classifiers & time-series estimators
3. Serialization: Save trained objects as .pkl files
4. Active Scoring: Query risk values directly from UI

4.2.1 Customer Churn Predictor

- **Problem:** Identify active customers who are at risk of leaving (churning) so Customer Success can reach out to them.
- **Model:** A Random Forest classifier trained on customer segments, purchase recency, order frequency, total spending, support ticket count, and average satisfaction score (CSAT).
- **Business Output:** The model assigns a churn probability (0% to 100%) to every active customer, which is displayed on the dashboard for immediate action.

4.2.2 Revenue Time-Series Forecaster

- **Problem:** Forecast weekly sales revenue for the next 12 weeks to help with cash flow planning.
- **Model:** Autoregressive Ridge Regression.
- **Features:** Calendar indicators (week-of-year seasonality), trend growth indices, weekly sales lag variables ($t - 1$, $t - 2$, $t - 3$, $t - 4$), and a 4-week moving average.
- **Business Output:** Generates a 90-day forward trend line displaying projected revenue targets.

4.2.3 Supplier Logistics Risk Classifier

- **Problem:** Predict if a hardware shipment will arrive late.
- **Model:** Random Forest classifier.
- **Features:** Carrier, supplier ID, product category, and destination warehouse.
- **Business Output:** Estimates delay risk probabilities for pending shipments, allowing managers to adjust carrier contracts.